NUST
NATIONAL UNIVERSITY
OF SCIENCES & TECHNOLOGY

EE-437

Digital Design Verification

Project Report

Register File Verification with

UVM RAL Model

Group Member	CMS-ID
Moonis Amir Zaman (Leader)	365753
Muhammad Bilal Khan	380046

Class: BEE-13

Course Instructor: Syed Jawad Hussain Shah

Lab Engineer: Adnan Aslam

School of Electrical Engineering and Computer Science, National University of Sciences and Technology Islamabad, Pakistan

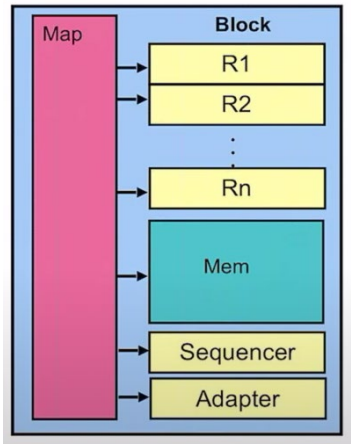
(2025)

NUST
NATIONAL UNIVERSITY
OF SCIENCES & TECHNOLOGY

Introduction

- **Goal:** To verify the functionality of a Register File using the UVM Register Abstraction Layer (RAL)
- **Key Objectives:**
    - Develop a UVM testbench with a robust register model
    - Implement comprehensive test sequences for register access
    - Verify both front-door (bus-based) and back-door (direct) register accesses
    - Ensure data integrity and correct register behavior
    - Collect functional coverage to measure verification completeness

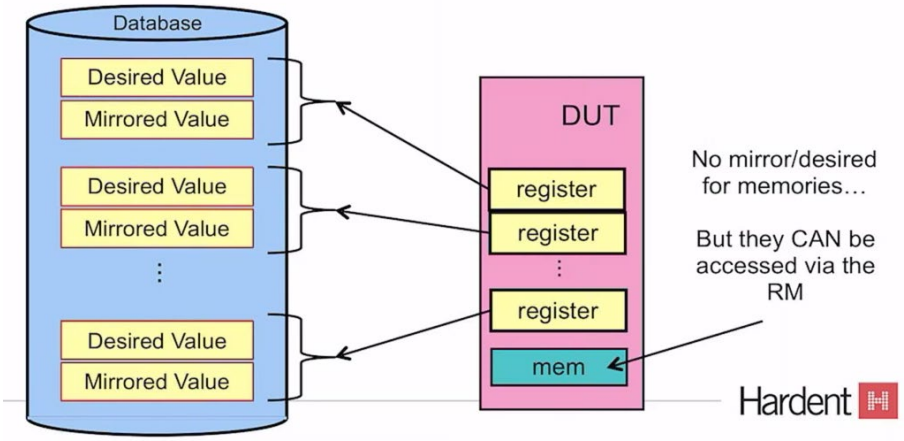
Register Model



(credit: Introduction to the UVM Register Layer, Hardent, Inc.)

- UVM provides base classes to build a "register model"
    - Script generated, executable model, based on design HW/SW spec
    - Represents "just" the storage elements in your design
    - Convenient access to registers & memories of DUT via one or more bus interfaces
    - Provides a convenience layer for accessing DUT registers and memories

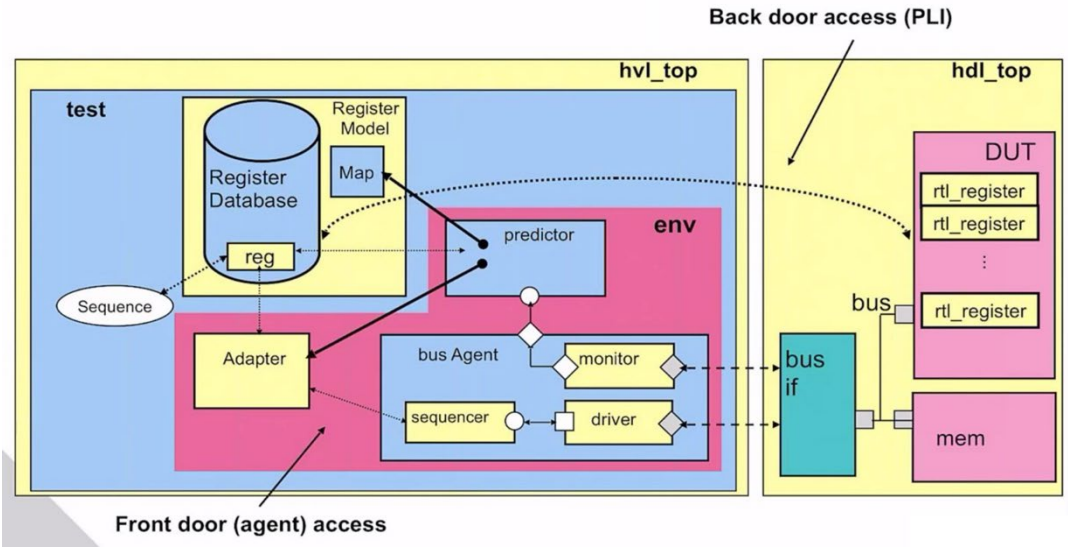
Register Database



(credit: Introduction to the UVM Register Layer, Hardent, Inc.)

- Register model has its own 'database'
- Represents the state of the hardware registers
- Each DUT register is represented twice!
- **Desired Value** – what we WANT in the hardware reg next
- **Mirrored Value** – what we THINK is in the hardware reg now

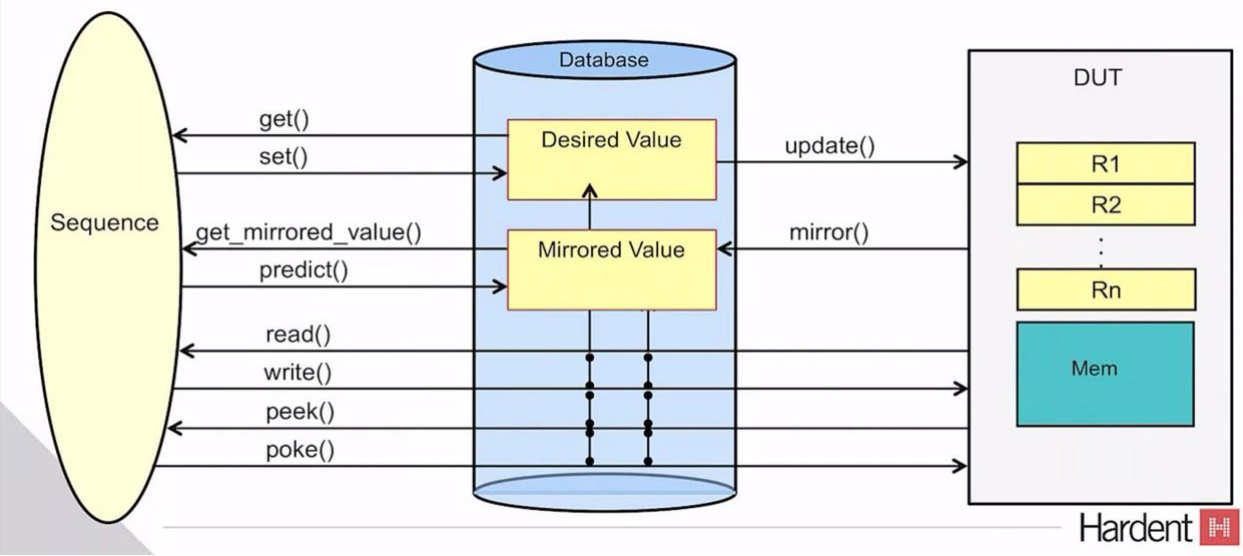
The UVM Register Abstraction Layer (RAL)



(credit: Introduction to the UVM Register Layer, Hardent, Inc.)

- **What is RAL?**
    - A UVM library providing a standardized, object-oriented way to model and access DUT registers.
    - Abstracts low-level bus protocol details.
- **Core Components:**
    - **Test:** Orchestrates the verification, selects sequences.
    - **Environment (RegisterFile_env):**
        - ❑ Integrates Agent, Register Model, Predictor, Adapter.
    - **Agent (RegisterFile_agent):**
        - ❑ **Sequencer:** Manages sequence execution.
        - ❑ **Driver:** Drives bus transactions to the DUT via RegisterFile_if.
        - ❑ **Monitor:** Observes bus activity, collects transactions for coverage and prediction.
    - **Interface (RegisterFile_if):** Connects TB to DUT.
    - **DUT (RegisterFile):** The hardware design under test.
- **RAL Integration is Key:** The Register Model, Adapter, and Predictor enable high-level register interaction.
- **Key RAL Concepts:**
    - **Register Database (Model):** Software representation of DUT registers.
        - ❑ **Desired Value:** What the test *intends* to write.
        - ❑ **Mirrored Value:** Software's "shadow" of the hardware state, updated by prediction.
    - **Access Methods:**
        - ❑ read() / write(): Front-door (bus) operations.
        - ❑ peek() / poke(): Back-door (direct memory/register) operations.
        - ❑ get() / set(): Interact with the Desired Value.
        - ❑ get_mirrored_value() / predict(): Interact with the Mirrored Value.
        - ❑ update() / mirror(): Synchronize model with DUT.

Access Methods



(credit: Introduction to the UVM Register Layer, Hardent, Inc.)

Front-Door Vs. Back-Door Access

Backdoor Access	Frontdoor Access
Take zero simulation time	Use a bus transaction which will take at least one RTL clock cycle
Write direction accesses force the HW register bits to the specified value	Write direction accesses do a normal HW write
Read direction accesses return the current value of the HW register bits	Read direction accesses to a normal HW read, data is returned using the HW data path
In the UVM register model, backdoor accesses are always auto-predicted - the mirrored value reflects the HW value	Frontdoor accesses are predicted based on what the bus monitor observes
Only the register bits accessed are affected, side effects may occur when time advances depending on HW implementation	Side effects are modelled correctly
By-passes normal HW	Simulates real timing and event sequences, catches errors due to unknown interactions

(credit: Introduction to the UVM Register Layer, Hardent, Inc.)

- **Front-Door Access:**
  - Uses the DUT's actual bus interface (read(), write()).
  - Involves driver, sequencer, and DUT bus logic.
  - Takes simulation time (reflects real hardware timing).

  - Verifies the entire data path and control logic.
- **Back-Door Access:**
  - Directly accesses DUT registers/memory via simulator mechanisms (e.g., PLI, VPI) (peek(), poke()).
  - Typically takes zero simulation time.
  - Bypasses bus interface logic.
  - Useful for:
    - ❑ Initializing DUT state quickly.
    - ❑ Checking internal register values without bus cycles.
    - ❑ Testing registers in isolation.
- **Both are crucial for comprehensive verification.**

Register Model Implementation

- **Individual Registers:**
  - Each DUT register (e.g., rxtx0, ctrl) modeled as a **uvm_reg** class.
  - Fields within registers defined as **uvm_reg_field** with attributes (size, LSB, access, reset).
- **Register Block:**
  - Container (**uvm_reg_block**) for all register objects.
  - Defines a **uvm_reg_map**:
  - Maps register objects to their addresses in the DUT.
  - Specifies bus width and endianness.
  - Configures HDL paths for back-door access.
- **Adapter:**
  - Translates generic RAL operations to bus-specific sequence items and vice-versa.
    - ❑ **reg2bus()**: RAL operation -> Bus transaction.
    - ❑ **bus2reg()**: Bus transaction -> RAL operation (for predictor).

NUST
NATIONAL UNIVERSITY
OF SCIENCES & TECHNOLOGY

Verification Plan: Test Sequences

- **Primary Test Sequence:**
  - Comprehensive Register Testing: Iterates through all registers in the model.
  - For each register:
  - **Predefined Patterns:** (0x0000, 0xFFFF_FFFF, 0xAAAA_AAAA, etc.)
    - ❑ Front-Door Write -> Front-Door Read -> Back-Door Peek. Verify at each step.
  - **Random Value (Front-Door):**
    - ❑ Front-Door Write (random data) -> Front-Door Read -> Back-Door Peek. Verify.
  - **Random Value (Back-Door):**
    - ❑ Back-Door Poke (random data) -> Front-Door Read -> Back-Door Peek. Verify.
  - **Field-Level Access Tests:**
    - ❑ Targeted specific fields (e.g., ss_reg.char_len).
    - ❑ Verified using uvm_reg_field::write() and uvm_reg_field::poke().
    - ❑ Confirmed field value and impact on the full register.

Test Case ID	Test Case Name	Description	UVM Sequence(s) Used	Key Features Verified	Stimulus Details	Expected Outcome	Priority	Status
REG_ACCESS								
Pattern test	Comprehensive RAL Access	Verifies R/W via FD & BD for all registers and fields using various data patterns. Checks model synchronization.	`RegisterFile_reg_seq`	VREG_RW, VREG_BD, VREG_FD, VREG_DATA, VREG_RAL_SYNC	All registers; patterns (0, all 1s, walking, random); FD/BD R/W;	All RAL operations (write/read/poke/peek/field ops) complete with UVM_IS_OK. Read/peeked data matches written/poked data. Mirrored values are correct.	High	Implemented
STRESS_TESTING								
Randomized test	RAL Random Stress Test	Subjects registers to a high volume of randomized read/write operations via FD.	Custom random sequence in `RegisterFile_reg_seq`	VREG_RW, VREG_DATA, VREG_RAL_SYNC	N (large number) random FD R/W operations to random valid addresses with random data.	All operations complete with UVM_IS_OK. Data integrity maintained. Model synchronized. No unexpected DUT behavior.	High	Implemented
FIELD_TESTING								
ss_reg field	RAL Field Test	Write/Poke JUST the char_len field of ss_reg	Test case 4 (label) in `RegisterFile_reg_seq`	VREG_FIELD	Change character field of ss_reg	Only the char_len field should be modified	Low	Implemented

NUST
NATIONAL UNIVERSITY
OF SCIENCES & TECHNOLOGY

RAL: Advanced Field Access with set() and update()

- **Challenge in Test Case 4 (Field Writes):**
    - Initial attempts to use uvm_reg_field::write() for front-door access to fields (e.g., ss_reg.char_len) sometimes led to unexpected full register values being written to the DUT.
    - This was due to uvm_reg_field::write() performing a Read-Modify-Write (RMW) on the entire register, where the "read" part (base value for other fields) wasn't always reflecting the immediate DUT state as intuitively expected by the sequence logic.
- **uvm_reg_field::set(value):**
    - **Purpose:** To precisely define the **desired value** for only the target field within the software register model, leaving other fields' desired states untouched initially.
    - **Action:** Updates the model's internal "desired state" for that specific field, which in turn contributes to the overall desired state of the parent register.
    - **Impact:** No immediate hardware write.
- **uvm_reg::update(status) (on the parent register):**
    - **Purpose:** To reliably synchronize the hardware register with the model's overall desired state, especially after targeted field modifications using set().
    - **Action for Field Writes:**
        1. update() compares the parent register's (e.g., ss_reg) full desired value (now incorporating the field value set by char_len_field.set()) with its mirrored value.
        2. If different, it initiates a front-door write of the *entire calculated desired register value* to the DUT. This ensures other fields are written based on their (potentially previously mirrored or set) desired states.
    - **Benefit in Test Case 4:** Using char_len_field.set('ha) followed by reg_model.ss_reg.update(status) provided a more robust and predictable way to ensure the intended field modification occurred correctly within the context of the entire register being written to the DUT. This approach gives more explicit control over the RMW base by leveraging the model's desired state.

Code Snippets: Register File DUT

```
package shared_pkg;
    parameter REG_WIDTH = 32;
    parameter REG_DEPTH = 7;
    parameter ADDR_SIZE = 32;
    int clk_cycle = 4;

        typedef enum logic [ADDR_SIZE-1:0] {
        rxtx0_addr = 32'h0000_0000,
        rxtx1_addr = 32'h0000_0004,
        rxtx2_addr = 32'h0000_0008,
        rxtx3_addr = 32'h0000_000c,
        ctrl_addr = 32'h0000_0010,
        divider_addr = 32'h0000_0014,
        ss_addr = 32'h0000_0018} reg_e;

endpackage : shared_pkg


// From UVM cookbook

module RegisterFile(RegisterFile_if.DUT regif);
        import shared_pkg::*;

        logic [REG_WIDTH-1:0] rxtx0;  // Based on SPI register names
        logic [REG_WIDTH-1:0] rxtx1;
        logic [REG_WIDTH-1:0] rxtx2;
        logic [REG_WIDTH-1:0] rxtx3;
        logic [REG_WIDTH-1:0] ctrl;
        logic [REG_WIDTH-1:0] divider;
        logic [REG_WIDTH-1:0] ss;      // This register is tested for fields

    always @(posedge regif.clk) begin
        regif.valid = 0;
        if(regif.RegWrite) begin
            case (regif.addr)
                32'h0000_0000: rxtx0   <=      regif.write_data;
                32'h0000_0004: rxtx1   <=      regif.write_data;
                32'h0000_0008: rxtx2   <=      regif.write_data;
                32'h0000_000c: rxtx3   <=      regif.write_data;
                32'h0000_0010: ctrl    <=      regif.write_data;
                32'h0000_0014: divider <=      regif.write_data;
                32'h0000_0018: ss      <=      regif.write_data;
            endcase
            regif.valid = 1;
        end else begin
                case (regif.addr)
                32'h0000_0000: regif.read_data <= rxtx0;
                32'h0000_0004: regif.read_data <= rxtx1;
                32'h0000_0008: regif.read_data <= rxtx2;
                32'h0000_000c: regif.read_data <= rxtx3;
                32'h0000_0010: regif.read_data <= ctrl;
                32'h0000_0014: regif.read_data <= divider;
                32'h0000_0018: regif.read_data <= ss;
```

```
            endcase
            regif.valid = 1;
        end
    end

endmodule
```

We used the register file module based on the *UVM Cookbook* RAL chapter. The register names are based on the SPI protocol. But module functionality is just a register file.

Code Snippets: Register Write Sequence

```
package RegisterFile_write_sequence_pkg;
        import shared_pkg::*;
        import uvm_pkg::*;
        import RegisterFile_seq_item_pkg::*;
        `include "uvm_macros.svh"

        class RegisterFile_write_sequence extends uvm_sequence
#(RegisterFile_seq_item);
                `uvm_object_utils(RegisterFile_write_sequence)

                RegisterFile_seq_item seq_item;

                function new(string name = "RegisterFile_write_sequence");
                        super.new(name);
                endfunction : new

                task body();
                        repeat(500) begin
                                seq_item =
RegisterFile_seq_item::type_id::create("seq_item");
                                start_item(seq_item);
                                assert(seq_item.randomize());
                                finish_item(seq_item);
                        end
                endtask : body
        endclass : RegisterFile_write_sequence

endpackage : RegisterFile_write_sequence_pkg
```

Code Snippets: Register Class Extensions

```
package Registers_pkg;
        import uvm_pkg::*;
        `include "uvm_macros.svh"

        // Based on UVM cookbook
        class rxtx0 extends uvm_reg;
```

```
        /*------------------------------------------------------------------
        -- Interface, port, fields
        ------------------------------------------------------------------*/
                uvm_reg_field reserved;
                rand uvm_reg_field acs;
                rand uvm_reg_field ie;
                rand uvm_reg_field lsb;
                rand uvm_reg_field tx_neg;
                rand uvm_reg_field rx_neg;
                rand uvm_reg_field go_bsy;
                uvm_reg_field reserved2;
                rand uvm_reg_field char_len;

        /*------------------------------------------------------------------
        -- UVM Factory register
        ------------------------------------------------------------------*/
                // Provide implementations of virtual methods such as
get_type_name and create
                `uvm_object_utils(rxtx0)

        /*------------------------------------------------------------------
        -- Functions
        ------------------------------------------------------------------*/
                // Constructor
                function new(string name = "rxtx0");
                        super.new(name,32,UVM_NO_COVERAGE);
                endfunction : new

                virtual function void build();
                        // Build the fields using the factory
                        reserved = uvm_reg_field::type_id::create("reserved");
                        acs      = uvm_reg_field::type_id::create("acs");
                        ie       = uvm_reg_field::type_id::create("ie");
                        lsb      = uvm_reg_field::type_id::create("lsb");
                        tx_neg   = uvm_reg_field::type_id::create("tx_neg");
                        rx_neg   = uvm_reg_field::type_id::create("rx_neg");
                        go_bsy   = uvm_reg_field::type_id::create("go_bsy");
                      reserved2 = uvm_reg_field::type_id::create("reserved2");
                        char_len = uvm_reg_field::type_id::create("char_len");

                        // Configure the fields using the configure method
inherited from the uvm_field class
                        // field_object.configure(parent_register, size, lsb,
access, volatility, reset_value, 1, 1, 0)
                        reserved.configure              (this, 18, 14, "RO", 0,
1'b0, 1, 1, 0);
                        acs.configure                   (this, 1 , 13, "RW", 0,
1'b0, 1, 1, 0);
                        ie.configure                    (this, 1 , 12, "RW", 0,
1'b0, 1, 1, 0);
                        lsb.configure                   (this, 1 , 11, "RW", 0,
1'b0, 1, 1, 0);
                        tx_neg.configure                (this, 1 , 10, "RW", 0,
1'b0, 1, 1, 0);
```

```
                        rx_neg.configure                (this, 1 , 9 , "RW", 0,
1'b0, 1, 1, 0);
                        go_bsy.configure                (this, 1 , 8 , "RW", 0,
1'b0, 1, 1, 0);
                        reserved2.configure             (this, 1 , 7 , "RO", 0,
1'b0, 1, 1, 0);
                        char_len.configure              (this, 7 , 0 , "RW", 0,
1'b0, 1, 1, 0);

                endfunction : build

        endclass : rxtx0

        class rxtx1 extends uvm_reg;

        /*--------------------------------------------------------------------
        -- Interface, port, fields
        --------------------------------------------------------------------*/
                uvm_reg_field reserved;
                rand uvm_reg_field acs;
                rand uvm_reg_field ie;
                rand uvm_reg_field lsb;
                rand uvm_reg_field tx_neg;
                rand uvm_reg_field rx_neg;
                rand uvm_reg_field go_bsy;
                uvm_reg_field reserved2;
                rand uvm_reg_field char_len;

        /*--------------------------------------------------------------------
        -- UVM Factory register
        --------------------------------------------------------------------*/
                // Provide implementations of virtual methods such as
get_type_name and create
                `uvm_object_utils(rxtx1)

        /*--------------------------------------------------------------------
        -- Functions
        --------------------------------------------------------------------*/
                // Constructor
                function new(string name = "rxtx1");
                        super.new(name,32,UVM_NO_COVERAGE);
                endfunction : new

                virtual function void build();
                        // Build the fields using the factory
                        reserved = uvm_reg_field::type_id::create("reserved");
                        acs      = uvm_reg_field::type_id::create("acs");
                        ie       = uvm_reg_field::type_id::create("ie");
                        lsb      = uvm_reg_field::type_id::create("lsb");
                        tx_neg   = uvm_reg_field::type_id::create("tx_neg");
                        rx_neg   = uvm_reg_field::type_id::create("rx_neg");
                        go_bsy   = uvm_reg_field::type_id::create("go_bsy");
                      reserved2 = uvm_reg_field::type_id::create("reserved2");
                        char_len = uvm_reg_field::type_id::create("char_len");
```

```
                        // Configure the fields using the configure method
inherited from the uvm_field class
                        // field_object.configure(parent_register, size, lsb,
access, volatility, reset_value, 1, 1, 0)
                        reserved.configure     (this, 18, 14, "RO", 0, 1'b0, 1,
1, 0);
                        acs.configure          (this, 1 , 13, "RW", 0, 1'b0, 1,
1, 0);
                        ie.configure           (this, 1 , 12, "RW", 0, 1'b0, 1,
1, 0);
                        lsb.configure          (this, 1 , 11, "RW", 0, 1'b0, 1,
1, 0);
                        tx_neg.configure       (this, 1 , 10, "RW", 0, 1'b0, 1,
1, 0);
                        rx_neg.configure       (this, 1 , 9 , "RW", 0, 1'b0, 1,
1, 0);
                        go_bsy.configure       (this, 1 , 8 , "RW", 0, 1'b0, 1,
1, 0);
                        reserved2.configure    (this, 1 , 7 , "RO", 0, 1'b0, 1,
1, 0);
                        char_len.configure     (this, 7 , 0 , "RW", 0, 1'b0, 1,
1, 0);

                endfunction : build

        endclass : rxtx1

// SIMILAR uvm_reg EXTENSIONS FOR rxtx2, ss, divider etc…
```

Code Snippets: Register Block Class Extensions

```
package RegisterFile_block_pkg;
        import uvm_pkg::*;
        import Registers_pkg::*;
        `include "uvm_macros.svh"

        class RegisterFile_block extends uvm_reg_block;

        /*--------------------------------------------------------------------
        -- Interface, port, fields
        --------------------------------------------------------------------*/
                rand rxtx0 rxtx0_reg;
                rand rxtx1 rxtx1_reg;
                rand rxtx2 rxtx2_reg;
                rand rxtx3 rxtx3_reg;
                rand ctrl ctrl_reg;
                rand divider divider_reg;
                rand ss ss_reg;

                uvm_reg_map RegisterFile_map;
        /*--------------------------------------------------------------------
        -- UVM Factory register
        --------------------------------------------------------------------*/
```

```
		// Provide implementations of virtual methods such as
get_type_name and create
		`uvm_object_utils(RegisterFile_block)

	/*------------------------------------------------------------------
	-- Functions
	------------------------------------------------------------------*/
		// Constructor
		function new(string name = "RegisterFile_block");
			super.new(name, UVM_CVR_ALL);
		endfunction : new

		virtual function void build();
			rxtx0_reg = rxtx0::type_id::create("rxtx0_reg");
			rxtx0_reg.configure(this, null, "");
			rxtx0_reg.build();
			// HDL path to the register that will be used for back
door accesses and will be appended to the hierarchical HDL path to the module
that contains the implementation
			rxtx0_reg.add_hdl_path_slice("rxtx0", 0, 32); //
Default for last argument is "RTL"

			rxtx1_reg = rxtx1::type_id::create("rxtx1_reg");
			rxtx1_reg.configure(this, null, "");
			rxtx1_reg.build();
			// HDL path to the register that will be used for back
door accesses and will be appended to the hierarchical HDL path to the module
that contains the implementation
			rxtx1_reg.add_hdl_path_slice("rxtx1", 0, 32); //
Default for last argument is "RTL"

			rxtx2_reg = rxtx2::type_id::create("rxtx2_reg");
			rxtx2_reg.configure(this, null, "");
			rxtx2_reg.build();
			// HDL path to the register that will be used for back
door accesses and will be appended to the hierarchical HDL path to the module
that contains the implementation
			rxtx2_reg.add_hdl_path_slice("rxtx2", 0, 32); //
Default for last argument is "RTL"

			rxtx3_reg = rxtx3::type_id::create("rxtx3_reg");
			rxtx3_reg.configure(this, null, "");
			rxtx3_reg.build();
			// HDL path to the register that will be used for back
door accesses and will be appended to the hierarchical HDL path to the module
that contains the implementation
			rxtx3_reg.add_hdl_path_slice("rxtx3", 0, 32); //
Default for last argument is "RTL"

			ctrl_reg = ctrl::type_id::create("ctrl_reg");
			ctrl_reg.configure(this, null, "");
			ctrl_reg.build();
			// HDL path to the register that will be used for back
door accesses and will be appended to the hierarchical HDL path to the module
that contains the implementation
```

```
                        ctrl_reg.add_hdl_path_slice("ctrl", 0, 32); // Default
for last argument is "RTL"

                        divider_reg = divider::type_id::create("divider_reg");
                        divider_reg.configure(this, null, "");
                        divider_reg.build();
                        // HDL path to the register that will be used for back
door accesses and will be appended to the hierarchical HDL path to the module
that contains the implementation
                        divider_reg.add_hdl_path_slice("divider", 0, 32); //
Default for last argument is "RTL"

                        ss_reg = ss::type_id::create("ss_reg");
                        ss_reg.configure(this, null, "");
                        ss_reg.build();
                        // HDL path to the register that will be used for back
door accesses and will be appended to the hierarchical HDL path to the module
that contains the implementation
                        ss_reg.add_hdl_path_slice("ss", 0, 32); // Default for
last argument is "RTL"

                        RegisterFile_map = create_map("RegisterFile_map",
'h0000_0000, 4, UVM_LITTLE_ENDIAN, 1); // name, base, nBytes

                RegisterFile_map.add_reg(rxtx0_reg,    'h0000_0000, "RW");  //
reg, offset, access
                RegisterFile_map.add_reg(rxtx1_reg,    'h0000_0004, "RW");
                RegisterFile_map.add_reg(rxtx2_reg,    'h0000_0008, "RW");
                RegisterFile_map.add_reg(rxtx3_reg,    'h0000_000c, "RW");
                RegisterFile_map.add_reg(ctrl_reg,     'h0000_0010, "RW");
                RegisterFile_map.add_reg(divider_reg,  'h0000_0014, "RW");
                RegisterFile_map.add_reg(ss_reg,       'h0000_0018, "RW");

                // Specifying the hdl path to the module that contains the
implementation of the registers
                        add_hdl_path("RegisterFile_top.DUT", "RTL");

                        lock_model();
                endfunction : build

        endclass : RegisterFile_block

endpackage : RegisterFile_block_pkg
```

The `hdl_slice_path` configuration is essential for setting how many bits of each register can be accessed via backdoor.

Covergroup Configuration

```
// Functional Coverage Group
        covergroup RegisterFile_CG with function sample(RegisterFile_seq_item
trans);
            option.per_instance = 1;
            option.name = "MonitorCoverageInstance";

            // Coverpoint for register addresses accessed
            cp_addr : coverpoint trans.addr {
                bins rxtx0_access = {rxtx0_addr};
                bins rxtx1_access = {rxtx1_addr};
                bins rxtx2_access = {rxtx2_addr};
                bins rxtx3_access = {rxtx3_addr};
                bins ctrl_access  = {ctrl_addr};
                bins div_access   = {divider_addr};
                bins ss_access    = {ss_addr};
            }

            // Coverpoint for operation type (Read/Write)
            cp_reg_write : coverpoint trans.RegWrite {
                bins read_op  = {0};
                bins write_op = {1};
            }

            // Coverpoint for a specific field in a register, e.g., char_len
in ss_addr
            // This requires sampling the data value when the address is
ss_addr
            cp_ss_char_len : coverpoint trans.write_data[6:0] iff (trans.addr
== ss_addr && trans.RegWrite == 1) {
                bins len_0_7   = {[0:7]};    // Example: cover lengths 0 to 7
                bins len_8_15  = {[8:15]};
                bins len_max   = {7'h7F};    // Max char_len (7 bits)  i.e.,
7'b111_1111
            }

            // Coverpoint for valid signal status
            cp_valid_state : coverpoint trans.valid {
                bins op_valid = {1};
                //bins op_not_valid = {0};
            }

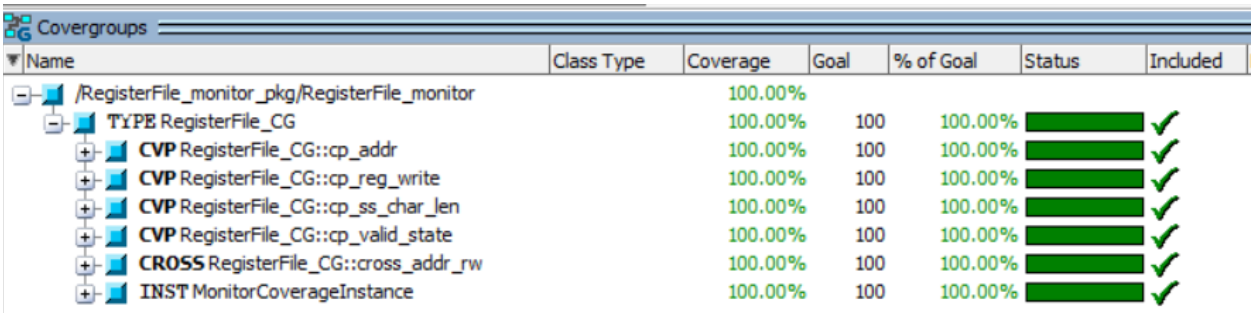
            // Cross coverage example: Read vs Write for each address
            cross_addr_rw : cross cp_addr, cp_reg_write;

        endgroup : RegisterFile_CG
```

The covergroup is defined and constructed in the monitor class.

NUST
NATIONAL UNIVERSITY
OF SCIENCES & TECHNOLOGY

Functional Coverage Results



All covergroups set are satisfied.

Code Coverage Report Snippet

```
Coverage Report by instance with details

================================================================================
====
=== Instance: /RegisterFile_top/RegisterFileif
=== Design Unit: work.RegisterFile_if
================================================================================
====
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                        141       139         2    98.58%

================================Toggle
Details================================

Toggle Coverage for instance /RegisterFile_top/RegisterFileif --

                                                 Node       1H->0L       0L->1H
"Coverage"
                                                 --------------------------------
--------------------------------
                                                 addr                 ENUM type
Value       Count
                                                valid              0            0
0.00

Total Node Count     =         74
Toggled Node Count   =         73
Untoggled Node Count =          1

Toggle Coverage      =      98.58% (139 of 141 bins)

================================================================================
====
```

```
=== Instance: /RegisterFile_top/DUT
=== Design Unit: work.RegisterFile
================================================================================
====
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                        18        16         2    88.88%

================================Branch
Details================================

Branch Coverage for instance /RegisterFile_top/DUT

    Line         Item                      Count     Source
    ----         ----                      -----     ------
  File ../RTL/RegisterFile.sv
-----------------------------------IF Branch--------------------------------
----
    16                                       122     Count coming in to IF
    16              1                         44
    27              1                         78
Branch totals: 2 hits of 2 branches = 100.00%

-----------------------------------CASE Branch------------------------------
------
    17                                        44     Count coming in to CASE
    18              1                          6
    19              1                          6
    20              1                          6
    21              1                          6
    22              1                          6
    23              1                          6
    24              1                          8
                                           ***0***     All False Count
Branch totals: 7 hits of 8 branches = 87.50%

-----------------------------------CASE Branch------------------------------
------
    28                                        78     Count coming in to CASE
    29              1                         11
    30              1                         11
    31              1                         11
    32              1                         11
    33              1                         11
    34              1                         11
    35              1                         12
                                           ***0***     All False Count
Branch totals: 7 hits of 8 branches = 87.50%


Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                      18        18         0   100.00%
```

```
================================Statement
Details================================

Statement Coverage for instance /RegisterFile_top/DUT --

    Line         Item                      Count     Source
    ----         ----                      -----     ------
  File ../RTL/RegisterFile.sv
    14              1                        122
    15              1                        122
    18              1                          6
    19              1                          6
    20              1                          6
    21              1                          6
    22              1                          6
    23              1                          6
    24              1                          8
    26              1                         44
    29              1                         11
    30              1                         11
    31              1                         11
    32              1                         11
    33              1                         11
    34              1                         11
    35              1                         12
    37              1                         78
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                        448       444         4    99.10%

================================Toggle
Details================================

Toggle Coverage for instance /RegisterFile_top/DUT --

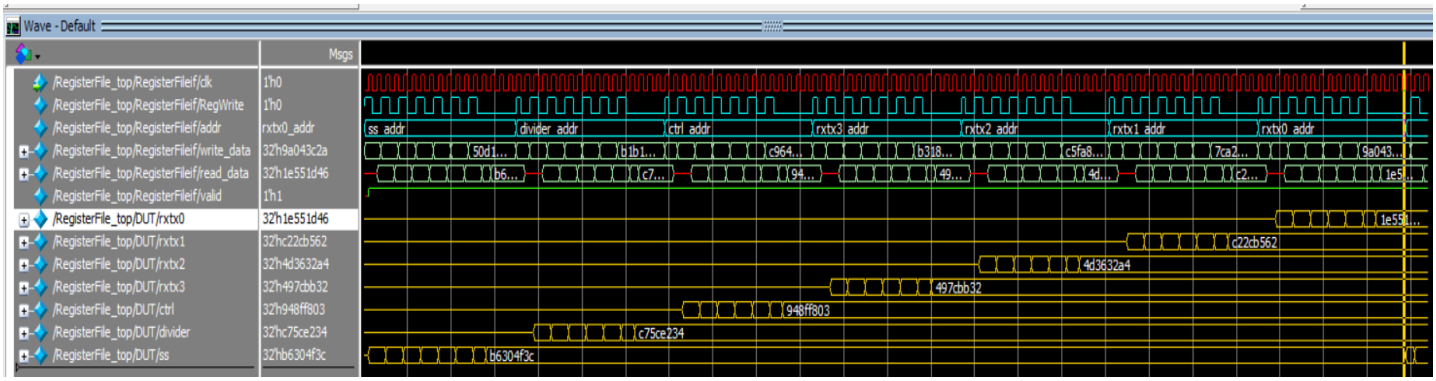
                                             Node      1H->0L      0L->1H
"Coverage"
                                             --------------------------
--------------------------------
                                    divider[17]           1           0
50.00
                                      rxtx0[17]           1           0
50.00
                                       rxtx2[1]           1           0
50.00
                                      rxtx3[17]           1           0
50.00

Total Node Count     =        224
Toggled Node Count   =        220
Untoggled Node Count =          4

Toggle Coverage      =      99.10% (444 of 448 bins)
```

NUST
NATIONAL UNIVERSITY
OF SCIENCES & TECHNOLOGY

Waveform Simulation



This waveform illustrates the UVM RAL-based verification of the DUT. The top signals show the clock (clk), bus control signals (RegWrite, addr, write_data, read_data, valid) driven by the UVM driver and responded to by the DUT. We observe a sequence of write and read operations targeting different registers (ss_addr, divider_addr, ctrl_addr, rxtx3_addr, etc.) with various data values. The lower signals (/RegisterFile_top/DUT/rxtx0 through /ss) represent the internal state of the DUT's registers, visibly updating one clock cycle after a corresponding write operation on the bus interface when RegWrite is high and valid asserts. For example, when addr is ss_addr and a write occurs, the /DUT/ss signal changes to the write_data value on the next rising clock edge, confirming successful register updates as orchestrated by the UVM RAL test sequences.

Console Output Snippet

```
# UVM_INFO verilog_src/uvm-1.1d/src/reg/uvm_reg_map.svh(1867) @ 480: reporter
[uvm_reg_map] Writing 'h1e551d0a at 'h18 via map
"reg_model.RegisterFile_map"...
# UVM_INFO ../UVM_Testbench/RegisterFile_driver.sv(40) @ 484:
uvm_test_top.env.agt.driver [run_phase] RegWrite = 0x1, addr = ss_addr,
write_data = 0x1e551d0a, read_data = 0xxxxxxxxx, valid = 0x1
# UVM_INFO verilog_src/uvm-1.1d/src/reg/uvm_reg_predictor.svh(218) @ 484:
uvm_test_top.env.RegisterFile_predictor [REG_PREDICT] Observed WRITE
transaction to register reg_model.ss_reg: value='h1e551d0a : updated value =
'h1e551d0a
# UVM_INFO ../UVM_Testbench/RegisterFile_monitor.sv(86) @ 484:
uvm_test_top.env.agt.monitor [run_phase] RegWrite = 0x1, addr = ss_addr,
write_data = 0x1e551d0a, read_data = 0x1e551d46, valid = 0x1
# UVM_INFO verilog_src/uvm-1.1d/src/reg/uvm_reg_map.svh(1914) @ 484: reporter
[uvm_reg_map] Wrote 'h1e551d0a at 'h18 via map "reg_model.RegisterFile_map":
UVM_IS_OK...
# UVM_INFO @ 484: reporter [RegModel] Wrote register via map
reg_model.RegisterFile_map: reg_model.ss_reg=0x1e551d0a
# UVM_INFO verilog_src/uvm-1.1d/src/reg/uvm_reg_map.svh(1994) @ 484: reporter
[uvm_reg_map] Reading address 'h18 via map "reg_model.RegisterFile_map"...
```

```
# UVM_INFO ../UVM_Testbench/RegisterFile_driver.sv(40) @ 488:
uvm_test_top.env.agt.driver [run_phase] RegWrite = 0x0, addr = ss_addr,
write_data = 0x00000000, read_data = 0x1e551d0a, valid = 0x1
# UVM_INFO verilog_src/uvm-1.1d/src/reg/uvm_reg_predictor.svh(223) @ 488:
uvm_test_top.env.RegisterFile_predictor [REG_PREDICT] Observed READ
transaction to register reg_model.ss_reg: value='h1e551d0a
# UVM_INFO ../UVM_Testbench/RegisterFile_monitor.sv(86) @ 488:
uvm_test_top.env.agt.monitor [run_phase] RegWrite = 0x0, addr = ss_addr,
write_data = 0x1e551d0a, read_data = 0x1e551d0a, valid = 0x1
# UVM_INFO verilog_src/uvm-1.1d/src/reg/uvm_reg_map.svh(2042) @ 488: reporter
[uvm_reg_map] Read 'h1e551d0a at 'h18 via map "reg_model.RegisterFile_map":
UVM_IS_OK...
# UVM_INFO @ 488: reporter [RegModel] Read  register via map
reg_model.RegisterFile_map: reg_model.ss_reg=1e551d0a
# UVM_INFO ../UVM_Testbench/RegisterFile_reg_seq.sv(216) @ 488:
uvm_test_top.env.agt.sqr@@reg_write_seq [RegisterFile_reg_seq] [ss_reg field
test] Actual full DUT value after RMW (read back): 0x000000001e551d0a
# UVM_INFO ../UVM_Testbench/RegisterFile_reg_seq.sv(238) @ 488:
uvm_test_top.env.agt.sqr@@reg_write_seq [RegisterFile_reg_seq] Finished
comprehensive register access test sequence
# UVM_INFO ../UVM_Testbench/RegisterFile_test.sv(43) @ 488: uvm_test_top
[run_phase] ****************************************reg_write_seq Sequence
ended****************************************
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1243) @ 488:
reporter [TEST_DONE] All end-of-test objections have been dropped. Calling
stop tasks
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 488:
reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_phase.svh(1199) @ 488: reporter
[PH/TRC/EXE/ALLDROP] Phase 'common.run' (id=93) PHASE EXIT ALL_DROPPED
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_phase.svh(1215) @ 488: reporter
[PH_READY_TO_END] Phase 'common.run' (id=93) PHASE READY TO END
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_phase.svh(1215) @ 488: reporter
[PH_READY_TO_END] Phase 'uvm.uvm_sched.post_shutdown' (id=304) PHASE READY TO
END
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 1004
# UVM_WARNING :    0
# UVM_ERROR :    0
# UVM_FATAL :    0
# ** Report counts by id
# [PH/TRC/EXE/ALLDROP]     1
# [PHASESEQ]    13
# [PH_READY_TO_END]    13
# [Questa UVM]     4
# [REG_PREDICT]   122
# [RNTST]     1
# [RegMem]   126
# [RegModel]   175
# [RegisterFile_reg_seq]    99
# [TEST_DONE]     2
# [run_phase]   232
# [uvm_reg_map]   216
```

```
# ** Note: $finish    : C:/questasim64_2024.1/win64/../verilog_src/uvm-
1.1d/src/base/uvm_root.svh(430)
#    Time: 488 ns  Iteration: 61  Instance: /RegisterFile_top
```

Note that only the last few output statements are shown here, for test case 4 (ss_reg field test), as described in the verification plan previously.

The complete code files and reports are provided alongside this report.

Conclusion

This project successfully demonstrated a comprehensive UVM RAL-based verification strategy for a Register File DUT, validating both front-door and back-door register accesses, including individual field manipulations. Through systematic test sequences, the integrity of register read/write operations and the synchronization between the hardware and the UVM register model (via desired and mirrored values updated by the predictor) were confirmed. Functional coverage metrics for register addresses, operation types, and specific field values provided quantitative feedback on the thoroughness of the verification, highlighting the effectiveness of UVM RAL in creating robust, reusable, and abstract register test environments.